

HelixLLM

HelixLLM

Un binario, seis modos: inferencia compatible con OpenAI y Anthropic, desde tu portátil hasta un clúster multi-host.

Resumen

HelixLLM es un sistema distribuido de LLM de nivel empresarial desarrollado en Go: un único binario con un sistema de modos que escala desde el desarrollo en un solo host hasta la producción en múltiples hosts. Ofrece APIs totalmente compatibles con OpenAI y Anthropic sobre HTTP/3, con inferencia local mediante llama.cpp, una cadena de respaldo multi-proveedor con puntuación, un pipeline de RAG y un sistema de agentes ReAct.

Descripción breve

HelixLLM es un sistema distribuido de LLM basado en un único binario y en Go. Expone APIs compatibles con OpenAI y Anthropic sobre HTTP/3, ejecuta inferencia local con llama.cpp, descubre y puntúa automáticamente proveedores en la nube gratuitos para formar una cadena de conmutación por error, e incorpora un pipeline de conocimiento RAG y un agente ReAct con capacidad de invocación de herramientas, desplegable en seis modos distintos.

Descripción detallada

HelixLLM es un sistema distribuido de LLM de nivel empresarial construido en Go con Gin, y su principal ventaja es que un único artefacto sirve para cualquier escala. Se compila en un solo binario cuyo sistema de modos decide en el momento del despliegue qué función desempeñará: ejecutarlo como full para una instancia todo-en-uno en un portátil, o distribuir las responsabilidades entre los modos gateway, brain, knowledge, agents

y control en múltiples hosts. El mismo código, reorganizado en lugar de reescrito, desde la máquina de un desarrollador hasta un clúster de producción.

Domina dos dialectos con fluidez: APIs totalmente compatibles con OpenAI y Anthropic, de modo que los clientes SDK existentes de ambos ecosistemas funcionan sin modificaciones, todo servido sobre HTTP/3 (QUIC) con respaldo automático a HTTP/2 y TLS 1.3. La inferencia local se ejecuta mediante llama.cpp con soporte para CUDA, Metal y ROCm, lo que permite que la misma compilación se acelere en hardware Nvidia, Apple y AMD por igual. Lo más destacado es la cadena de respaldo multi-proveedor, que convierte la conocida falta de fiabilidad de la inferencia en la nube gratuita en un recurso gestionado y autorreparable: HelixLLM descubre automáticamente modelos gratuitos de más de 7 proveedores en la nube (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together), los puntúa mediante LLMsVerifier con una actualización cada 5 minutos y enruta las solicitudes a través de la cadena ordenada con conmutación automática ante errores 429/5xx, siempre con llama.cpp local como último recurso garantizado, de modo que una petición nunca falla simplemente por falta de un proveedor disponible.

Más allá de la inferencia pura, HelixLLM es una plataforma de aplicaciones completa: incluye un pipeline de conocimiento RAG (ingesta, fragmentación, incrustación, búsqueda vector) y un sistema de agentes ReAct con invocación de herramientas, sesiones de conversación e integración con RAG, todo en el mismo binario. El sistema de modos también optimiza la comunicación: en modo `full`, todas las capas se comunican mediante llamadas directas en proceso Go sin sobrecarga de red, mientras que el mismo binario, distribuido en varios hosts, se coordina mediante gRPC, SSE y Kafka. Para completar el conjunto, ofrece negociación de contenido Brotli/gzip, transmisión SSE que coincide byte a byte con los formatos OpenAI y Anthropic, autenticación con claves API y JWT con limitación de tasa, métricas Prometheus, trazabilidad OpenTelemetry y un amplio conjunto de submódulos Go para infraestructura de producción.

Por qué lo creamos

Los equipos necesitan inferencia portátil, compatible con estándares y resiliente, sin tener que reescribir clientes ni depender de un único proveedor o máquina. HelixLLM se desarrolló para que el mismo binario pueda ejecutarse localmente en desarrollo y escalar a un clúster de producción con múltiples hosts, utilizando los dialectos OpenAI y Anthropic que ya emplean los clientes.

Por qué es un cambio radical

Consolida toda una pila de inferencia —puerta de enlace, inferencia local, respaldo en la nube, RAG y agentes— en un solo binario controlado por un interruptor de modo, de modo que la arquitectura que despliegues sea una decisión en tiempo de ejecución, no un proyecto de replataformización. Además, convierte en una ventaja lo que antes era un problema: la fiabilidad de los proveedores en la nube pasa a ser una preocupación de primer orden, gestionada mediante una cadena de respaldo autosanable y puntuada que reevalúa a los proveedores cada pocos minutos y siempre degrada a inferencia local garantizada. La capacidad que esto desbloquea es un único punto final en el que realmente puedes confiar: compatible con estándares, portátil desde el portátil hasta el clúster e incapaz de fallar porque un proveedor aguas arriba limite las solicitudes o falle.

Qué tiene de innovador

- Un único binario con un sistema de seis modos que funciona todo en uno o como roles distribuidos —llamadas directas en proceso a Go en modo `full`, `gRPC/SSE/Kafka` cuando está dividido—, de modo que la topología de despliegue cambia sin modificar el código ni incurrir en un coste de red no deseado.
- Una cadena de respaldo multiproveedor puntuada y de autodescubrimiento entre más de 7 proveedores gratuitos, clasificados continuamente mediante `LLMsVerifier` con conmutación automática por fallos `429/5xx` y un último recurso garantizado con `llama.cpp` —la capacidad de nivel gratuito convertida en capacidad fiable—.
- Superficies compatibles tanto con OpenAI *como* con Anthropic, servidas sobre `HTTP/3` con respaldo automático a `HTTP/2`, para que los clientes de ambos ecosistemas se conecten sin modificaciones.
- Inferencia local que abarca `CUDA`, `Metal` y `ROCm` desde un único código base —el mismo *build* se ejecuta acelerado en hardware Nvidia, Apple y AMD—.

Principales desafíos técnicos y cómo los resolvimos

- **Escalar de un host a muchos sin reescribir.** La mayoría de los sistemas imponen una barrera rígida entre “desarrollo local” y “producción distribuida”, y cruzarla implica reestructurar. Eliminamos esa barrera con un sistema de modos en un solo binario: las mismas capas se comunican mediante llamadas directas en proceso en modo `full` y

cambian de forma transparente a gRPC/SSE/Kafka en modos distribuidos, de modo que escalar es un cambio de configuración, no un puerto.

- **Proveedores en la nube gratuitos, poco fiables y con límites de tasa.** La inferencia de nivel gratuito es rápida hasta que devuelve un 429 o desaparece a mitad de una solicitud. La hicimos fiable mediante el autodescubrimiento de modelos disponibles, su puntuación con LLMsVerifier, el seguimiento proactivo de cabeceras de límite de tasa para redirigir el tráfico antes de que un proveedor estrangule las solicitudes, y la conmutación automática por fallos a lo largo de la cadena clasificada hasta llegar a llama.cpp local —así, la inestabilidad del *pool* nunca afecta al llamante—.
- **Compatibilidad con clientes de dos ecosistemas.** Reescribir clientes para adoptar un nuevo *backend* de inferencia es inviable. Implementamos tanto las formas API de OpenAI como de Anthropic —hasta sus distintos formatos de transmisión SSE—, de modo que los SDK de ambos entornos apunten a HelixLLM y funcionen sin más.

Pila tecnológica

- **Go + Gin** — elegidos porque un runtime de binario único y concurrencia prioritaria es lo que hace posible todo el sistema de modos: una sola compilación que puede funcionar como servidor local o como rol en un clúster. Incluye el sistema completo y la capa HTTP de la pasarela.
- **HTTP/3 (QUIC) + TLS 1.3, con retroceso a HTTP/2** — seleccionados por su transporte moderno, de baja latencia y resiliente a fallos de conexión, expuesto como superficie del servidor con negociación automática para que los clientes que no soporten QUIC retrocedan silenciosamente a HTTP/2.
- **llama.cpp (CUDA/Metal/ROCm)** — elegido para la inferencia local portátil que acelera en backends de Nvidia, Apple y AMD desde un único código base; además, actúa como proveedor de último recurso garantizado para evitar que la cadena de retroceso falle por completo.
- **LLMsVerifier** — seleccionado para convertir “qué proveedor es bueno ahora” en un número; evalúa y clasifica la cadena de retroceso en la nube cada cinco minutos, de modo que el enrutamiento se ajusta a la calidad en tiempo real, no a suposiciones obsoletas.
- **Proveedores en la nube (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together)** — elegidos para aprovechar la capacidad de los niveles gratuitos de múltiples fuentes; se descubren y clasifican automáticamente en una única cadena de conmutación por error para que ningún proveedor sea un punto único de fallo.
- **gRPC + SSE + Kafka** — seleccionados como protocolos de transporte entre modos para despliegues distribuidos: gRPC

para llamadas servicio a servicio, SSE para transmisión en tiempo real y Kafka para flujos de eventos desacoplados entre roles.

- **Almacén de vectores / embeddings** — elegido para impulsar el pipeline de conocimiento RAG de principio a fin: ingesta, segmentación, incrustación y búsqueda en documentos que fundamentan las respuestas del modelo.
- **Prometheus + OpenTelemetry** — seleccionados para métricas y trazado distribuido que siguen una solicitud a través de los modos desplegados.
- **Submódulos vasic-digital Go** — elegidos para reutilizar primitivas de infraestructura de producción ya probadas en lugar de reconstruirlas, manteniendo la coherencia de la base del sistema con el resto de la pila.

Estado y notas de transparencia

- **Estado: beta.** Sistema de inferencia distribuida funcional y en desarrollo activo.
- **Licencia: por determinar.** El repositorio no declara licencia en sus metadatos (`licenseInfo` nulo); esto NO ESTÁ VERIFICADO y debe resolverse antes de especificar una licencia.
- El repositorio canónico actualmente redirige a github.com/HelixDevelopment/llm; la ruta `HelixLLM` redirige a él. Las cifras de cobertura y número de submódulos en el README son autoinformadas.

Nivel de prioridad: Helix-principal.